WATER QUALITY AND POTABILITY PREDICTION

USING SUPERVISED MACHINE LEARNING

Mini Project Report :

Project ID: AI_20

Submitted by:
N. Sai Manvitha (24STUCHH010978)
S. Viplav Kumar (24STUCHH010922)

Bachelor of Technology in Computer Science and Engineering
Department of Artificial Intelligence
ICFAI UNIVERSITY.
Academic Year: 2026
Report submission date: 14 November 2026

Abstract

Safe drinking water depends on chemical and physical properties that are normally measured in laboratories, which is slow and costly for routine monitoring. This project applies supervised machine learning to predict whether a water sample is Potable (1) or Not Potable (0) from eight measurements: pH, Hardness, Solids, Chloramines, Sulfate, Conductivity, Organic Carbon and Trihalomethanes. The study uses the Kaggle Water Potability dataset, which contains 3,276 samples. Zero pH values were treated as missing, and median imputation and standard scaling were placed inside scikit-learn pipelines fitted on training data only. A stratified 80:20 split with random seed 42 produced 2,620 training and 656 testing samples.

Logistic Regression, Decision Tree and Random Forest were tuned using 5-fold stratified cross-validation on the training set, with F1 for the Potable class as the tuning metric and balanced class weights to handle the 61:39 imbalance.

Random Forest was selected on cross-validated training F1 (0.546). On the unseen test set it achieved accuracy 0.637, balanced accuracy 0.609 and ROC-AUC 0.673, but its F1 for the Potable class was 0.508. A trivial model that always predicts Potable scored 0.561 F1, so the selected model does not outperform that baseline on F1. Exploratory analysis showed that every feature has only a weak linear correlation with potability (absolute value at most 0.034), which explains the modest performance. The system is an educational prediction tool with a Streamlit

Introduction

Water quality monitoring is a basic requirement for public health. Drinking water is checked for parameters such as pH, dissolved solids, hardness, disinfectant residuals, sulfate, organic matter and disinfection by-products. Laboratory analysis of these parameters is accurate, but it requires trained staff, equipment and time, which limits how often water can be checked, especially in small communities.

Machine learning offers a complementary approach. If a model is trained on historical water samples whose potability is already known, it can learn relationships between the measured parameters and the label, and then estimate the label for a new sample. Such a model can support screening and prioritisation. It cannot replace the laboratory tests that determine legal drinking-water compliance.

This project is a supervised learning study built on a labelled dataset. It compares three standard classification algorithms under the same preprocessing and evaluation rules and presents the best of them through a simple web interface.

Problem Statement

Given the chemical and physical measurements of a water sample (pH, Hardness,
Solids, Chloramines, Sulfate, Conductivity, Organic Carbon and Trihalomethanes),
predict whether the sample is Potable (suitable for drinking) or Not Potable.
The problem is a binary classification task. The training data is imbalanced, with
about 61% Not Potable and 39% Potable samples, so the solution must be judged with
metrics that are fair under imbalance, not with accuracy alone.

Objectives

1. Understand the dataset through inspection, missing-value analysis, duplicate
   checks, class distribution analysis and exploratory data visualisation.
2. Build a leakage-free preprocessing pipeline with median imputation, standard
   scaling and a stratified 80:20 train-test split.
3. Train and tune three supervised classifiers (Logistic Regression, Decision Tree
   and Random Forest) using cross-validation on training data only.
4. Evaluate every model on training and testing data using accuracy, balanced
   accuracy, precision, recall, F1-score and ROC-AUC, and compare them against
   trivial baselines.
5. Select the best model using training-data cross-validation, save it, and
   deploy it in a web interface that validates inputs and displays the predicted
   class and probability.
6. State the limitations of the system clearly, including that it is an
   educational tool and not a drinking-water certification method.

Existing System and Proposed System

Existing system

Water potability is currently determined by laboratory analysis. Samples are collected from a source, sent to a testing facility, and analysed for each parameter, such as pH, hardness, dissolved solids, disinfectant residuals, sulfate, organic matter and disinfection by-products. The results are compared with national or international drinking-water standards. This approach is accurate and legally recognised, but it has three limitations: it is slow, it is costly to repeat frequently, and it provides no early estimate when a sample has not yet been sent for testing.

Proposed system

This project proposes a supervised machine learning classifier that learns from previously recorded water samples whose potability is already known. Given the eight measured parameters of a new sample, the system predicts whether the sample is Potable (1) or Not Potable (0), and it reports the probability of the Potable class. The proposed system is intended for educational use and for screening discussions. It does not replace laboratory testing or regulatory compliance.

Dataset Description

Source and licence

The dataset is the Water Potability dataset published on Kaggle by Aditya Kadiwal (https://www.kaggle.com/datasets/adityakadiwal/water-potability). The file used in this project is water_potability.csv.

Structure

The dataset contains 3,276 rows and 10 columns: nine water quality measurements and one target label. The columns are listed below.

Column	Type	Description
ph	float	pH of the water (0 to 14)
Hardness	float	Capacity to precipitate soap, mainly calcium and magnesium (mg/L)
Solids	float	Total dissolved solids (ppm)
Chloramines	float	Chloramines disinfectant level (ppm)
Sulfate	float	Dissolved sulfate (mg/L)
Conductivity	float	Electrical conductivity (microS/cm)
Organic_carbon	float	Total organic carbon (ppm)
Trihalomethanes	float	Trihalomethane level (microg/L)
Turbidity	float	Cloudiness measured by light scattering (NTU)
Potability	integer	Target label: 1 = Potable, 0 = Not Potable

Features used for modelling

The brief specifies eight input parameters, so the models use the first eight columns above. Turbidity is kept in the dataset and in the exploratory analysis, but it is excluded from modelling and from the web form. This is a deliberate

design decision that follows the project brief. Its correlation with potability
(0.002) is also negligible, which supports this choice.

Data quality checks

Check	Result
Duplicate rows	0
Missing values, ph	491 (15.0%)
Missing values, Sulfate	781 (23.8%)
Missing values, Trihalomethanes	162 (4.9%)
Rows with any missing value	1,265 (38.6%)
Zero values in ph	1 (physically implausible; treated as missing)

Target distribution

Class	Count	Percentage
0, Not Potable	1,998	61.0%
1, Potable	1,278	39.0%

The dataset is moderately imbalanced. A model that always predicts Not Potable
would be correct 61% of the time, which is why accuracy alone is not used to
judge the models.

Methodology

The project follows a standard supervised learning workflow:

1. Load and inspect the dataset (dimensions, data types, missing values, duplicates and class distribution).
2. Perform exploratory data analysis (distributions, boxplots, class distribution and correlations).
3. Clean the data with fixed, row-level rules.
4. Split the data into training (80%) and testing (20%) sets, stratified by class, with random seed 42.
5. Build a preprocessing and classification pipeline for each algorithm.
6. Tune each model with 5-fold stratified cross-validation on the training set only, using F1-score for the Potable class as the tuning metric.
7. Evaluate every model on the training set and the testing set, and compare the results with two trivial baselines.
8. Select the best model using cross-validated training F1, save it, and deploy it in a Streamlit web interface.

The test set is used only once, in step 7, after all tuning decisions were made.
This prevents the test results from influencing the model choice.

Algorithms compared

Logistic Regression: a linear model that estimates the probability of the Potable class from a weighted sum of the scaled features.
Decision Tree: a model that splits the data with a series of threshold rules on the features. Its depth and minimum leaf size control how closely it fits the training data.

Random Forest: an ensemble of 200 decision trees, each trained on a random
sample of the data and features. Predictions are made by averaging the trees.

Baselines

Two trivial models provide reference points. They ignore the input features:
- Always predict Not Potable (the majority class).
- Always predict Potable (the minority class).

These show what a model would score without learning anything from the data.

Hyperparameter grids

Logistic Regression: C in {0.01, 0.1, 1, 10}
Decision Tree: max_depth in {3, 5, 7, 10, None}; min_samples_leaf in {1, 10, 20}
Random Forest: max_depth in {5, 10, None}; min_samples_leaf in {1, 5}

Every model uses class_weight="balanced", so errors on the Potable class count
more during training.

Data Pre-Processing

Each preprocessing step has a specific reason.

1. Duplicate check. The dataset has no exact duplicate rows. The check remains
in the code so the pipeline works on any future version of the data.

2. Treating pH = 0 as missing. One sample has a pH of exactly 0. Drinking water
with a pH of 0 is physically implausible, so the value is most likely a data
entry or sensor error. It is replaced with a missing value and then imputed
like the other missing pH values.

3. Feature selection. Only the eight features in the brief are kept for modelling.
Turbidity is removed, as explained in Section 5.

4. Median imputation. The 1,265 rows with a missing value are not dropped,
because that would remove 38.6% of the data. Instead, each missing value is
replaced with the median of its column. The median is used rather than the
mean because it is not pulled by extreme values, and several features have
outliers (see Section 8). The medians are calculated from the training data
only. The training medians were: pH 7.035, Hardness 196.928, Solids 20,866.336,
Chloramines 7.118, Sulfate 333.074, Conductivity 424.941, Organic_carbon
14.215, Trihalomethanes 66.566.

5. Standard scaling. Each feature is rescaled to a mean of 0 and a standard

deviation of 1. This matters for Logistic Regression, which is sensitive to the size of each feature. Solids is measured in thousands while pH is below 15, so without scaling Solids would dominate the model. Tree-based models are not affected by scale, but the same pipeline is used for all three models so the code stays consistent.

6. Stratified train-test split. The data is divided 80:20 with stratification. The split produces 2,620 training rows and 656 testing rows. The class ratio is preserved in both sets:

Training: 60.99% Not Potable, 39.01% Potable
Testing: 60.98% Not Potable, 39.02% Potable

Preventing data leakage

Data leakage happens when information from the test set influences training. It makes results look better than they really are. This project prevents leakage in three ways:

- The split is made before any statistics are calculated.
- Imputation and scaling are inside a scikit-learn Pipeline, so they are fitted only on the training data. During cross-validation they are refitted inside each fold.
- The test set is not used for tuning or model selection.

The final check confirmed 0 missing values remained after preprocessing, and the array shapes were 2,620 x 8 for training and 656 x 8 for testing.

Exploratory Data Analysis

The exploratory analysis was run in eda_plots.py. It produced five figures,
saved in the figures folder.

Figure 1: Class distribution (01_class_distribution.png)



This bar chart shows 1,998 Not Potable and 1,278 Potable samples. It confirms
the imbalance and motivates the use of class weights and F1-score.

Figure 2: Missing values (02_missing_values.png)

This chart shows the percentage of missing values in each column. Only ph,
Sulfate and Trihalomethanes have missing values. Sulfate has the most, at 23.8%.

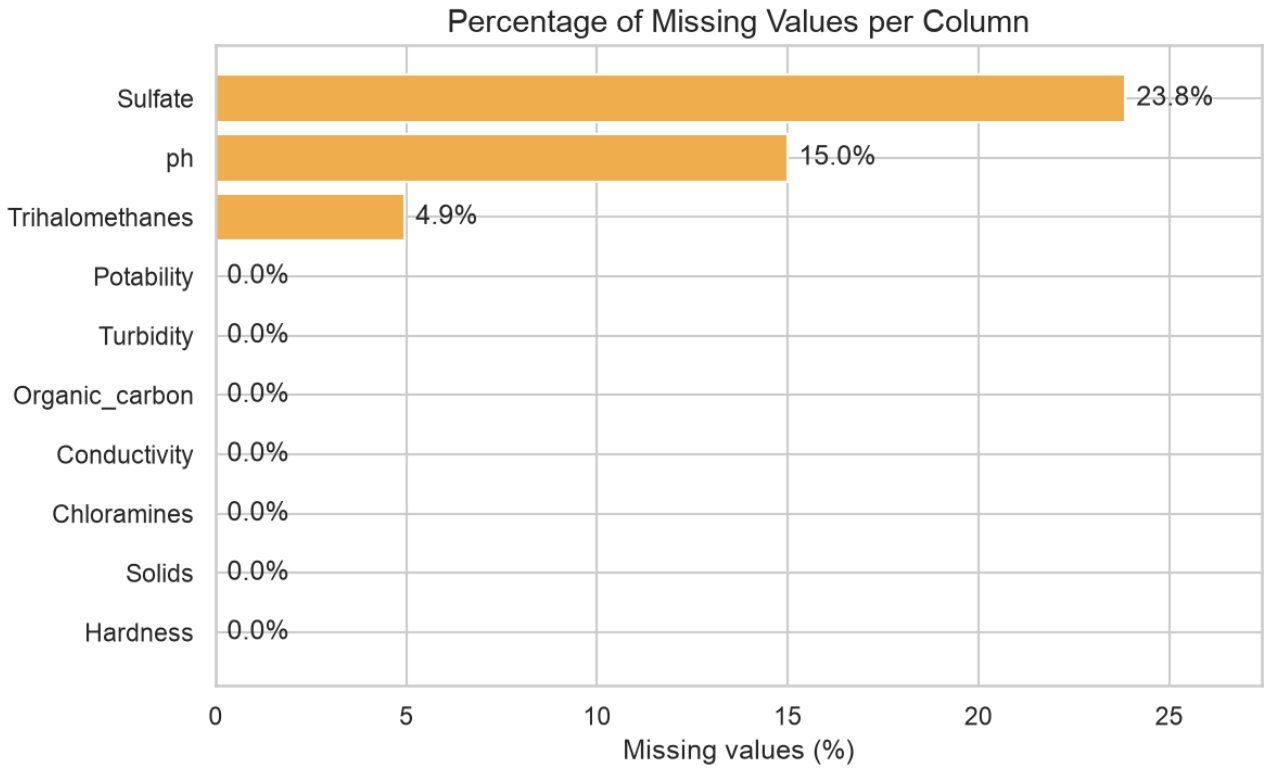
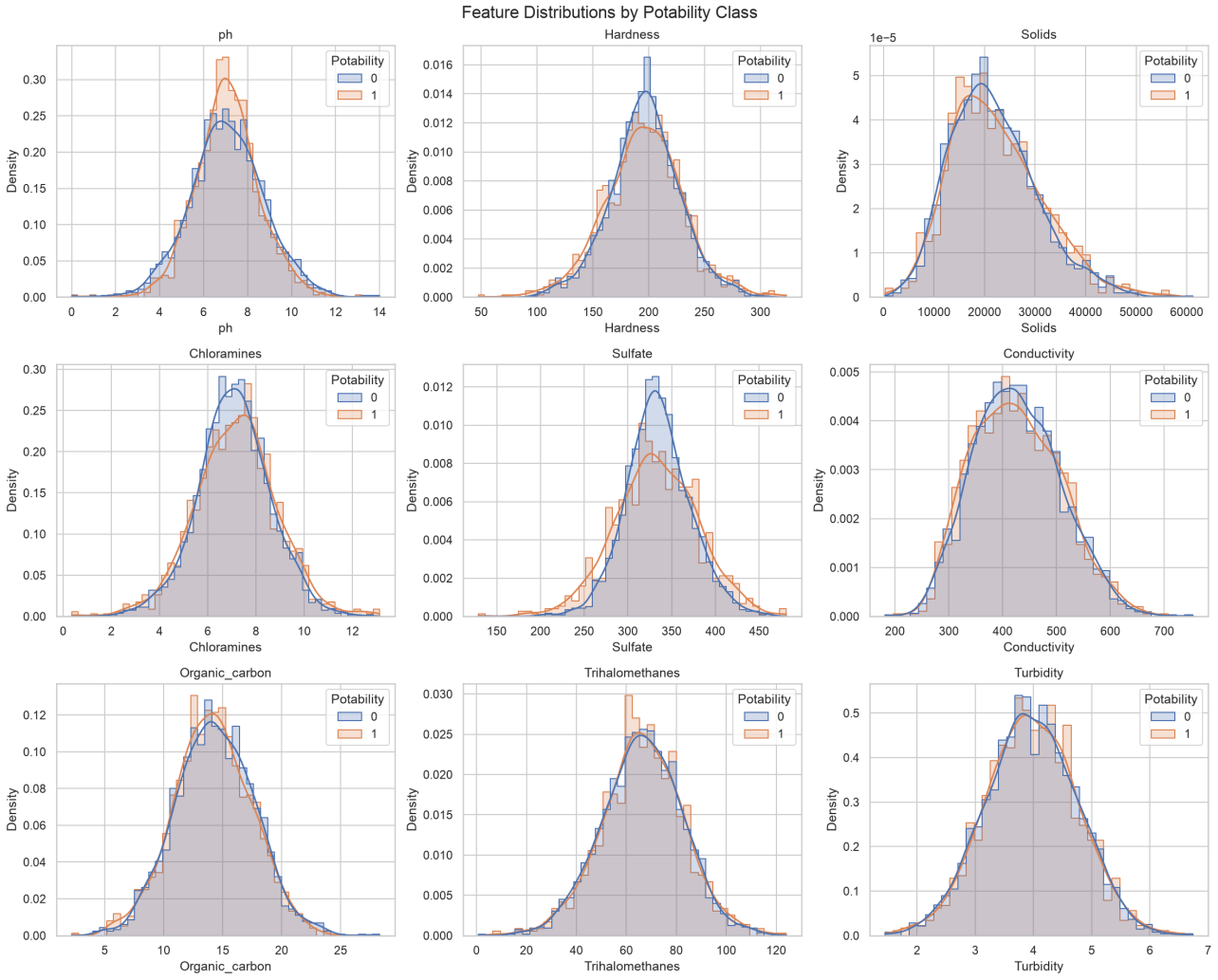


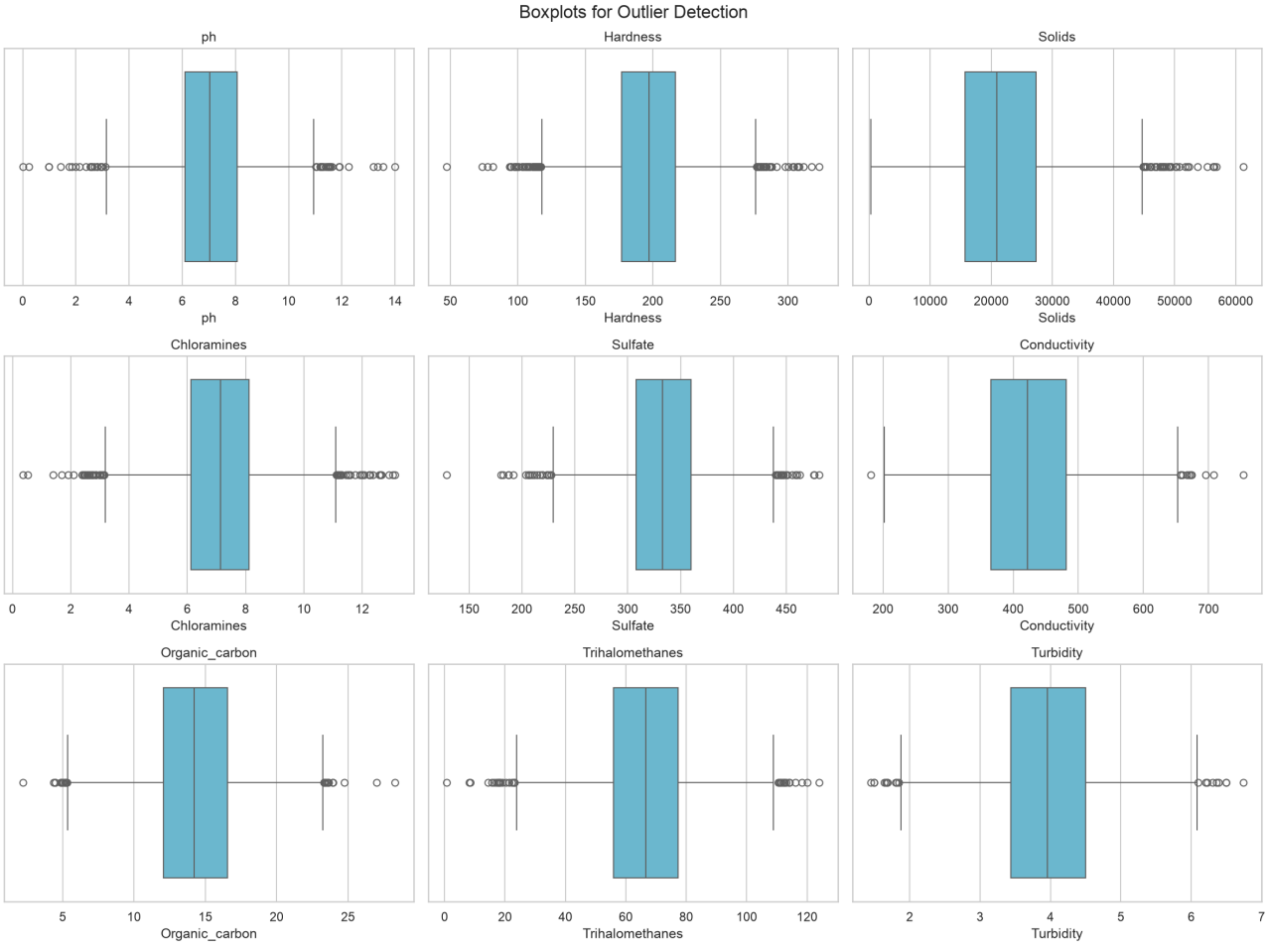
Figure 3: Feature distributions by class (03_feature_distributions.png)



Each histogram shows one feature, with the Potable and Not Potable distributions
overlaid. Most features overlap heavily between the two classes, which means
they do not separate potability clearly on their own. The clearest difference is
in Sulfate, where the Not Potable class has a sharper peak near 330 to 340 mg/L.
Solids and Trihalomethanes are slightly right-skewed.

Figure 4: Boxplots

Boxplots show the median, the spread and the outliers of each feature. Several
features have points beyond the whiskers on both sides, including pH, Hardness,
Chloramines, Sulfate, Organic_carbon, Trihalomethanes and Turbidity. Solids has
its outliers mainly above about 45,000 ppm. These values are plausible for real



water, so they were kept, except for the pH of 0 described in Section 7.

Figure 5: Correlation heatmap



The heatmap shows the correlation between each feature and the target, and
between features themselves.

Correlation with Potability (strongest first, in absolute value):

Solids	0.034
Organic_carbon	-0.030
Chloramines	0.024
Sulfate	-0.024

Hardness	-0.014
Conductivity	-0.008
Trihalomethanes	0.007
ph	-0.004
Turbidity	0.002

Correlation between features: the strongest pair is Solids and Sulfate at 0.172.
All other feature pairs are lower, so no two features are redundant.

Key finding
No feature has a strong linear relationship with potability. The largest
absolute correlation is 0.034. Correlation measures only straight-line
relationships, so it does not rule out nonlinear patterns. This is one reason
Random Forest was included. The weak correlations also explain why the models
achieve modest performance, which is discussed in Section 18.

Algorithms Used

Logistic Regression

Logistic Regression is a linear classifier. It computes a weighted sum of the
scaled input features and passes the result through a sigmoid function, which
converts it to a probability between 0 and 1. A sample is predicted as Potable
when this probability is at least 0.5. The parameter C controls regularisation,
which limits how large the weights can grow. Smaller values of C apply stronger
regularisation. The grid tested C = 0.01, 0.1, 1 and 10, and C = 1 was selected.

Decision Tree

A Decision Tree classifies a sample by asking a series of yes/no questions of the
form "Is Solids greater than 20,500?". At each step it chooses the split that
best separates the classes, measured by Gini impurity. Two settings control how
much the tree learns: max_depth limits the number of questions, and
min_samples_leaf sets the smallest number of training samples allowed in a final
group. The selected settings were max_depth = None (no depth limit) and
min_samples_leaf = 10.

Random Forest

A Random Forest combines many decision trees. Each of the 200 trees is trained on
a random bootstrap sample of the training data, and at each split it considers

only a random subset of the features. The forest predicts by averaging the
probabilities of all trees. Averaging reduces the tendency of single trees to
memorise their training data. The selected settings were
max_depth = None and
min_samples_leaf = 5.

Why these three algorithms

They form a progression from a simple linear model (Logistic Regression), to a
single nonlinear model (Decision Tree), to an ensemble of nonlinear models
(Random Forest). Comparing them shows whether extra model complexity helps on
this dataset. They are standard supervised classifiers that suit tabular data,
run quickly on a laptop, and are covered in standard machine learning courses.

Model Training

Each model is a Pipeline with two parts: preprocessing (median imputation and
standard scaling, from preprocessing.py), followed by the classifier. Placing the
preprocessing inside the pipeline means it is refitted on the training portion of
every cross-validation fold, so the validation portion never influences it.

Tuning method

Hyperparameters were chosen with GridSearchCV, which tests every combination in
the grid. Each combination was scored with 5-fold stratified cross-validation on
the 2,620 training rows. The score used was F1 for the Potable class. After
choosing the best combination, GridSearchCV refitted the model on all training
rows. The test set was not used at any point in this stage.

Results of tuning

Model	Best parameters	CV F1	Time
Logistic Regression	C = 1	0.4160	0.1 s
Decision Tree	max_depth = None, min_samples_leaf = 10	0.5366	0.6 s
Random Forest	max_depth = None, min_samples_leaf = 5	0.5463	4.1 s

Model selection rule

The model with the highest cross-validated F1 on the training data was selected.

That rule was fixed before the test set was examined. Random Forest had the
highest score (0.5463).

Training Performance

Training performance shows how well each fitted model predicts the samples it was trained on (2,620 rows). These scores are not a measure of generalisation. They are reported so the gap between training and testing can be seen.

Model	Accuracy	Bal. Acc.	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.520	0.516	0.406	0.498	0.448	0.514
Decision Tree	0.785	0.791	0.689	0.817	0.748	0.883
Random Forest	0.978	0.980	0.956	0.988	0.972	0.998
Baseline: always Not Potable	0.610	0.500	0.000	0.000	0.000	0.500
Baseline: always Potable	0.390	0.500	0.390	1.000	0.561	0.500

Interpretation

Random Forest scores 0.972 F1 and 0.998 ROC-AUC on the training data, which is close to perfect. Decision Tree scores 0.748 F1. Logistic Regression is near chance level, with ROC-AUC 0.514, which shows it cannot separate the classes with a straight-line boundary.

The large drop from training to testing for the tree models is called overfitting. The models have learned patterns specific to the training samples that do not carry over to new samples. Random Forest falls from 0.972 to 0.508 F1, and

Decision Tree falls from 0.748 to 0.519. Logistic Regression does not overfit,
because its training and testing scores are similar. Its problem is underfitting,
meaning the model is too simple for the pattern in the data.

Testing Performance

Testing performance is the main result. The 656 test samples were held back
during all tuning and selection, so these scores estimate how the models would
perform on new water samples.

Overall results on the test set

Model	Accuracy	Bal. Acc.	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.529	0.529	0.419	0.531	0.468	0.548
Decision Tree	0.587	0.584	0.476	0.570	0.519	0.636
Random Forest	0.637	0.609	0.540	0.480	0.508	0.673
Baseline: always Not Potable	0.610	0.500	0.000	0.000	0.000	0.500
Baseline: always Potable	0.390	0.500	0.390	1.000	0.561	0.500

Per-class classification report

Model	Class	Precision	Recall	F1	Support
Logistic Regression	Not Potable (0)	0.637	0.527	0.577	400
	Potable (1)	0.418	0.531	0.468	256
Decision Tree	Not Potable (0)	0.685	0.598	0.638	400
	Potable (1)	0.476	0.570	0.519	256
Random Forest	Not Potable (0)	0.689	0.738	0.713	400
	Potable (1)	0.539	0.480	0.508	256

Weighted average F1: Logistic Regression 0.535, Decision Tree 0.592, Random Forest 0.633.

Interpretation Random Forest has the highest accuracy (0.637), balanced accuracy (0.609) and
ROC-AUC (0.673). Its accuracy is 2.7 points above the "always Not Potable"
baseline, and its balanced accuracy is 10.9 points above the 0.500 chance level.
Its ROC-AUC means that, for a randomly chosen Potable sample and a randomly
chosen Not Potable sample, it ranks the Potable one higher about 67% of the time.

On the Potable class, Random Forest has the highest precision (0.539) but the
lowest recall of the two tree models (0.480). The Decision Tree has the highest
F1 for the Potable class (0.519), 0.011 higher than Random Forest. A difference
this small on 256 samples is within normal variation, so it does not change the
selection.

The "always Potable" baseline has the highest F1 (0.561) because it predicts
Potable for every sample, which gives recall of 1.0. This is an important
limitation: none of the trained models beats this baseline on F1 for the Potable
class.

Comparison of Models.

Ranking by test metrics

Metric	Best model	Comment
Accuracy	Random Forest (0.637)	Highest overall correctness
Balanced accuracy	Random Forest (0.609)	Fairest measure under imbalance
ROC-AUC	Random Forest (0.673)	Best ranking of samples
F1 (Potable)	Decision Tree (0.519)	Within 0.011 of Random Forest
Recall (Potable)	Logistic Regression (0.531)	Finds the most Potable samples

Why Random Forest was selected

The selection rule was cross-validated training F1, which chose Random Forest
(0.5463, against 0.5366 for Decision Tree and 0.4160 for Logistic Regression).
Test results support this choice. Random Forest is best on three of the four
measures that are fair under class imbalance (accuracy, balanced accuracy and
ROC-AUC), and it is within 0.011 F1 of the best model.

Switching to the Decision Tree because of its slightly higher test F1 would mean
choosing the model based on the test set, which is data leakage. The honest
conclusion is that the two tree models perform similarly, and Random Forest is
preferred for its better ranking and balanced accuracy.

Why Logistic Regression was not chosen

Logistic Regression has the lowest CV F1 and the lowest ROC-AUC (0.548). Its
results show that a linear boundary does not capture the data well.

Confusion Matrices and Graphs.

A confusion matrix counts how many test samples fall into each combination of
actual class and predicted class. The rows are the actual classes and the columns
are the predicted classes.

Confusion matrices on the test set (656 samples)

Random Forest (selected model)

	Predicted Not Potable	Predicted Potable
Actual Not Potable (400)	295	105
Actual Potable (256)	133	123

Decision Tree

	Predicted Not Potable	Predicted Potable
Actual Not Potable (400)	239	161
Actual Potable (256)	110	146

Logistic Regression

	Predicted Not Potable	Predicted Potable
Actual Not Potable (400)	211	189
Actual Potable (256)	120	136

These counts were derived from the reported precision, recall and support values.
They reproduce each model's reported test accuracy (Random Forest 418/656 = 0.637,
Decision Tree 385/656 = 0.587, Logistic Regression 347/656 = 0.529).

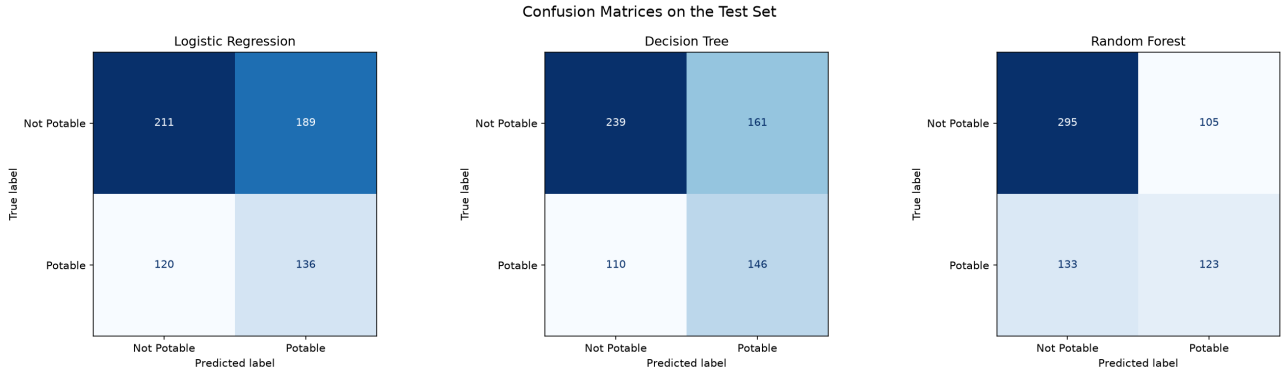
Reading the Random Forest matrix

- 295 Not Potable samples were correctly identified.
- 123 Potable samples were correctly identified.

- 105 Not Potable samples were predicted as Potable. This is the more serious
  error for a drinking-water application, because unsafe water would be labelled
  potable. It is 26% of the 400 Not Potable samples.
- 133 Potable samples were predicted as Not Potable. This is a false alarm. It
  is 52% of the 256 Potable samples, which is why recall is only 0.480.

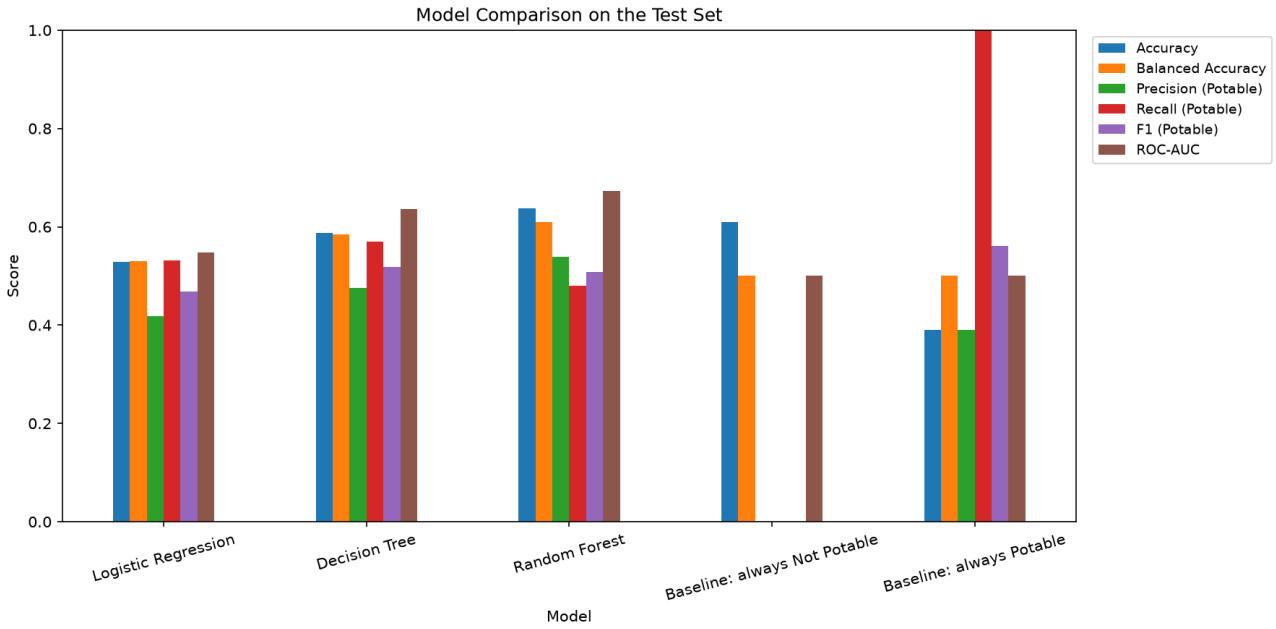
Figures

Figure 6: Confusion matrices on the test set



(Shows the matrices for all three models side by side. Use this figure to check
the counts above.

Figure 7: Model comparison on the test set



A grouped bar chart of accuracy, balanced accuracy, precision, recall, F1 and
ROC-AUC for every model and both baselines. It shows that the models sit between
the two trivial baselines on most metrics, and that none beats the "always
Potable" baseline on F1.

Final Prediction System

The final system is a Streamlit web application (app.py) that loads the saved
Random Forest pipeline (best_model.joblib) and predicts potability for a new
water sample.

How it works

1. The user enters the eight parameters: pH, Hardness, Solids, Chloramines,
   Sulfate, Conductivity, Organic Carbon and Trihalomethanes.
2. Each input is validated. The system rejects empty fields, non-numeric text,
   and values outside the range seen in the dataset. An error message is shown
   for each invalid field, and no prediction is made.
3. For valid input, the saved pipeline applies the same median imputation and
   scaling used in training, then predicts the class.
4. The page displays the predicted class (Potable or Not Potable), the model's
   probability of the Potable class, and the name of the selected model.
5. A disclaimer states that the tool is for educational use and does not certify
   drinking-water safety.

Screenshots

Figure 8: Prediction result (screenshots/02_website_prediction.png)
The web form with valid values entered and the prediction displayed.

Figure 9: Input validation (screenshots/03_website_validation.png)
The error message shown when an invalid value is entered.

Figure 10: Training and test results (screenshots/04_training_results.png)
The terminal output of the training run, including the test set results table.

Testing the application

The application was tested with the sample values listed in the README. It
produced a prediction for each valid input, and it rejected invalid inputs
with an error message. Changing one input value changed the output, which shows
that the saved pipeline responds to the entered values.

Results

Main result

Random Forest was selected because it had the highest cross-validated F1 on the
training data (0.5463). On the unseen test set it achieved accuracy 0.637,
balanced accuracy 0.609 and ROC-AUC 0.673. Its F1 for the Potable class was
0.508.

Interpreting the metrics

Accuracy alone is misleading for this dataset. Always predicting Not Potable gives

61.0% accuracy with no ability to find Potable samples. Always predicting Potable
gives an F1 of 0.561, because it finds every Potable sample, but it misclassifies
all 400 Not Potable samples. Balanced accuracy shows this clearly: both trivial
baselines score exactly 0.500, which is chance level.

Against this measure, Random Forest scores 0.609, which is above chance but not
by a large margin. Its ROC-AUC of 0.673 shows a modest ability to rank Potable
samples above Not Potable samples.

Why the performance is modest

The features carry only a weak signal for potability. The largest correlation with
the target is 0.034, and the class distributions overlap in almost every feature
(Section 8). A model cannot exceed the information present in its inputs, so
modest performance is the expected outcome for this dataset.

Overfitting

Random Forest reaches 0.972 F1 on training data and 0.508 on test data. The
model memorises the training samples, including their noise, and this does not
transfer to new samples. Decision Tree shows the same pattern to a lesser degree.
Restricting tree depth and leaf size helped only a little, which suggests the
limit is the information in the features rather than the model's flexibility.

Answering the objectives

- Dataset understood: dimensions, types, missing values, duplicates and class
  balance were checked, and the imbalance was handled with balanced class
  weights and F1-based evaluation.
- Leakage-free pipeline: built with pipelines, a stratified split and training-
  only statistics.
- Three classifiers trained and tuned: all three were tuned with 5-fold
  cross-validation on training data only.
- Evaluated on training and testing data: accuracy, balanced accuracy,
  precision, recall, F1, ROC-AUC, classification reports and confusion matrices.
- Model selected, saved and deployed: Random Forest was selected by training
  cross-validation, saved, and deployed in the web interface.
- Limitations stated: see Section 17.

Practical implications

In the test set, the selected model labelled 105 of 400 Not Potable samples as
Potable (26%). For drinking water, that is the more serious error, because unsafe
water could be predicted as safe. The model should therefore never be used as the
only check on water safety. It is a demonstration of how supervised learning
works on water quality data, not a safety tool.

Limitations

1. Weak predictive signal. The features have low correlation with potability,
  and the models achieve modest test performance. Test F1 for the Potable class
  (0.508) is below the always-Potable baseline (0.561).

2. Overfitting of the tree models. The large gap between training and test F1
means the models do not generalise as well as their training scores suggest.

3. Limited hyperparameter search. The grids were kept small on purpose to keep
the run fast and understandable. A wider search might change the results
slightly, but it is unlikely to close the gap caused by weak features.

4. Single train-test split. The results come from one 80:20 split with random
seed 42. Repeated splits or repeated cross-validation would give a more
reliable estimate of variation. This was not done because of the time limit.

5. Turbidity excluded. Turbidity was not used in modelling, following the eight
input parameters in the brief. Its correlation with potability is negligible,
so this choice is unlikely to matter much, but it was not tested.

6. Source and labelling. The dataset is a public Kaggle file. The labelling
process used to assign potability is not described in this project, so the
labels cannot be independently verified.

7. Missing data handled by imputation. About 38.6% of rows contain at least one
missing value. Median imputation is a simple method and may not reflect the
true values.

8. Not a safety certification. The model predicts a label learned from a dataset.

It does not test water and is not a substitute for accredited laboratory
analysis or regulatory compliance.

9. Platform tested. The project was run on macOS with Python 3.15.2. Other
operating systems and Python versions were not tested.

Conclusion

This project built and evaluated a supervised machine learning system for
predicting whether a water sample is Potable or Not Potable from eight chemical
and physical measurements. The work covered the full pipeline: dataset inspection,
exploratory analysis, leakage-free preprocessing, training and tuning of three
classifiers, evaluation on training and testing data, model selection, and a web
interface with input validation.

Random Forest was selected on training-data cross-validation and achieved the
best test accuracy, balanced accuracy and ROC-AUC among the three models. However,
the features carry only a weak signal, and the selected model does not beat the
always-Potable baseline on F1. These results are reported as found.

The project shows that a standard supervised pipeline can learn some signal from
water quality data, and it shows how important correct evaluation is when classes
are imbalanced. It is an educational system and must not be used to certify
drinking water.

- Use a larger or better-documented dataset with clearly defined labels.
- Choose the decision threshold based on the cost of each error. Lowering the threshold would catch more Potable samples, at the cost of more false alarms.
- Use repeated cross-validation and report confidence intervals for all metrics.
- Try gradient boosting methods, such as XGBoost or LightGBM, and compare them fairly under the same rules.
- Test whether including Turbidity improves the model (Option B in the design decisions).
- Add model explanation tools, such as feature importance or SHAP values, so users can see which inputs drive a prediction.
- Calibrate the predicted probabilities so that a stated probability matches the observed frequency.
- Deploy the web application online after the submission deadline.Future Enhancements

References

1. Kadiwal, A. Water Potability dataset. Kaggle. https://www.kaggle.com/datasets/adityakadiwal/water-potability (Accessed: [7 November 2026])

2. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

3. Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.

4. Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984).

Classification and Regression Trees. Wadsworth.

5. Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical
Learning: Data Mining, Inference, and Prediction (2nd ed.). Springer.

6. Fawcett, T. (2006). An introduction to ROC analysis. Pattern Recognition
Letters, 27(8), 861-874.

7. Streamlit documentation. https://docs.streamlit.io

8. pandas documentation. https://pandas.pydata.org

9. scikit-learn documentation. https://scikit-learn.org.